Theoretical HTML Interview Questions

Basic Level

1. **What is HTML?**

   HTML, which stands for HyperText Markup Language, is the standard language for creating web pages. It provides the structure and content of a webpage

2. **What is the difference between HTML and HTML5?**
3. **What is DOCTYPE and why is it used?**
4. **What is the difference between Semantic and Non-Semantic elements?**
5. **Name some commonly used semantic tags in HTML.**
6. **What is the difference between Block-level and Inline elements?**
7. **What are self-closing tags? Give examples.**

   Self-closing tags in HTML are tags that do not require a separate closing tag.

8. **What is the use of the `alt` attribute in the `<img>` tag?**

   The `alt` attribute in the `<img>` tag provides alternative text for an image, which is displayed when the image cannot be loaded for any reason

9. **How does form submission work in HTML?**

   HTML forms submit data to a server using a `form` element with input fields and a submit button.

10. **What is the use of the `<meta>` tag?**

    The `<meta>` tag defines metadata about an HTML document.

`<meta>` tags always go inside the <head> element, and are typically used to specify character set, page description, keywords, author of the document, and viewport settings.

Metadata will not be displayed on the page, but is machine parsable.

⚡ Intermediate Level

11. **What are the new elements introduced in HTML5?**
12. **For example:** `<section>`, `<article>`, `<header>`, `<footer>`, `<nav>`, `<aside>`
13. **What is the difference between** `id` **and** `class` **in HTML?**
14. **What are local storage and session storage in HTML?**

Local Storage:

- Data persists even after the browser is **closed and reopened**.
- Stored data is **domain-specific**.

Session Storage:

- Stores data only for the **duration of the page session**.
- Data is lost once the browser tab is **closed**.
- Also domain-specific, and **tab-specific** (data in one tab can't be accessed in another).

15. **What are** `data-` **attributes?**

`data-` attributes are custom attributes used to store extra information directly on HTML elements.
They are not visible to users but can be accessed easily via JavaScript.

```
const div = document.querySelector("div");

let userId = div.dataset.userId; // "123"

let role = div.dataset.role;
```

16. **How can you make a webpage mobile responsive using HTML?**
17. **Do** `<div>` **and** `<span>` **have any semantic meaning?**
18. **What is an iframe? What are its use cases?**

An iframe, short for inline frame, is an HTML element that embeds another HTML document within the current one

19. **What tags or attributes are used in HTML for accessibility?**
20. **For example:** `aria-label`, `role`, **etc.**

Ek responsive image add karo using HTML only.

```
<img src="photo.jpg" alt="Responsive" style="max-width: 100%; height: auto;">
```

CSS Interview Questions

Theoretical CSS Interview Questions

1. What is CSS and what is its purpose in web development?

CSS, or Cascading Style Sheets, is a stylesheet language used to control the visual presentation of web pages.

Purpose of CSS in Web Development:

1. Styling HTML elements

Layout control

Helps position elements (e.g., using Flexbox, Grid)

3. Responsive design

2. What is the difference between inline, internal, and external CSS?

Comparison Table:

Feature	Inline CSS	Internal CSS	External CSS
Location	Inside HTML element	Inside `<style>` tag	Separate `.css` file
Reusability	□ Not reusable	□□ Limited	□ Highly reusable

3. What are CSS selectors? Name some common types with examples.

CSS selectors are patterns used to select the HTML elements you want to style.

4. What is the difference between id and class selectors in CSS?
5. What is the CSS box model? Explain its components.
6. What is the difference between margin and padding?
7. What are pseudo-classes and pseudo-elements in CSS? Give examples.

In CSS, pseudo-classes and pseudo-elements are used to style specific states or parts of an element

Pseudo-Classes (:)

A pseudo-class targets an element based on its state or position.

Pseudo-Class	Description	Example
`:hover`	When the mouse is over the element	`a:hover { color: red; }`
`:focus`	When the element is focused (e.g., input field)	`input:focus { border: 1px solid blue; }`
`:nth-child(n)`	Selects the nth child of a parent	`li:nth-child(2) { font-weight: bold; }`
`:first-child`	Selects the first child of a parent	`p:first-child { color: green; }`
`:checked`	Targets checked input elements	`input:checked { background: yellow; }`

Pseudo-Elements (::)

A pseudo-element **lets you** style specific parts **of an element.**

Pseudo-Element	Description	Example
`::before`	**Inserts content** before **the element's content**	`h1::before { content: "* "; color: red; }`
`::after`	**Inserts content** after **the element's content**	`p::after { content: " □"; }`
`::first-line`	**Targets the** first line **of text**	`p::first-line { font-weight: bold; }`
`::first-letter`	**Styles the** first letter	`p::first-letter { font-size: 200%; }`

8. What is the z-index property and how does it work?
9. What is the difference between relative, absolute, fixed, and sticky positioning?

10. What is the purpose of the display property? Explain block, inline, and inline-block.

The `display` property determines how an HTML element is displayed **in the layout — i.e., whether it acts as a block, inline, flex container, grid, etc.**

1. `display: block;`

- Always starts on a **new line**.
- Can set **width**, **height**, **margin**, and **padding**

2. `display: inline;`

- **Does not start on a new line**.
- Cannot set **width** or **height**.
- Only **horizontal padding/margin** works (not vertical).

3. `display: inline-block;`

- Like `inline` — it flows inline with text.
- Like `block` — you **can set width, height, padding, and margin**.
- Useful for layouts that need to sit side-by-side but still allow box styling.

11. What is a CSS reset and why is it used?

A CSS reset **is a set of CSS rules used to** remove default browser styling **(like margins, paddings, fonts, etc.) from HTML elements.**

Why is CSS Reset Used?

- **Consistency**: Ensures a **consistent baseline** across all browsers.
- **Control**: Gives developers **full control** over how elements are styled.
- **Foundation**: Acts as a clean starting point for custom designs.

Ex-

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

12. What are CSS media queries and how are they used for responsive design?
13. What is the difference between rem and em units in CSS?
14. What is Flexbox and what are its main properties?

For the Flex Container:

Property	Description	Example
`display`	Enables flex mode	`display: flex;`
`flex-direction`	Direction of items (row, column)	`flex-direction: row;`
`flex-wrap`	Allows items to wrap to next line	`flex-wrap: wrap;`
`justify-content`	Aligns items horizontally (main axis)	`justify-content: center;`
`align-items`	Aligns items vertically (cross axis)	`align-items: center;`
`align-content`	Aligns rows of items when wrapped	`align-content: space-between;`

15. What is CSS Grid and how is it different from Flexbox?
16. What is the float property and how does it affect layout?
17. What is the purpose of the clear property in CSS?
18. What is specificity in CSS and how is it calculated?
19. What is the difference between visibility: hidden and display: none?
20. What are CSS variables and how do you use them?

CSS Variables **(also known as** custom properties**) allow you to store values (like colors, sizes, or fonts) in reusable variables — making your stylesheets more** dynamic**,** maintainable**, and** consistent**.**

```
:root {
 --primary-color: #ff5722;
 --font-size: 16px;
}
body {
 font-size: var(--font-size);
 color: var(--primary-color);
}
a:hover {
 color: var(--primary-color);
}
```

21. What's the difference between `min-width`, `max-width`, and `width`?

Property	Controls	Can Be Overridden By
`width`	Fixed/preferred size	`min-width`, `max-width`
`min-width`	Minimum allowed size	—
`max-width`	Maximum allowed size	

Practical CSS Interview Questions

1. Write CSS to center a div horizontally and vertically in the viewport.
2. Create a responsive navigation bar using CSS Flexbox.
3. Style a button with a hover effect that changes its background color and text color.
4. Write CSS to create a two-column layout using CSS Grid.
5. How would you style a contact form to make it visually appealing and responsive?
6. Write CSS to make an image responsive so it scales within its container.
7. Create a CSS rule to apply a shadow effect to a card component.
8. Write CSS to create a sticky footer that stays at the bottom of the page.
9. How would you style a table to have alternate row colors and a hover effect?
10. Fix this buggy CSS code:

    Css

.box { width: 200px; height: 100px; background-color blue; margin: 10px 20px 10px; }

11. Write CSS to create a dropdown menu that appears on hover.
12. Style a form input field with a focus effect that changes its border color.

```
<style>
  input[type="text"] {
    padding: 10px;
    border: 2px solid #ccc;
    border-radius: 4px;
    transition: border-color 0.3s ease;
  }

  input[type="text"]:focus {
    border-color: #007BFF; /* Blue border on focus */
    outline: none;         /* Remove default outline */
  }
</style>
```

13. Create a CSS animation to fade in an element when the page loads.
14. Write CSS to make a div take up 50% of the parent container's width and align it to the right.
15. How would you use CSS to create a responsive image gallery with 3 columns on desktop and 1 column on mobile?

1. Create a responsive square box that always remains a square on all screens

```
.box {
  width: 100%;
  padding-top: 100%; /* Creates 1:1 aspect ratio */
  background: blue;
}
```

---

2. Center a div in middle of page using Flexbox

```
body {
```

```
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}
```

---

3. Make a responsive 3-column layout using Flexbox or Grid

```
.container {
  display: flex;
  flex-wrap: wrap;
}
.column {
  flex: 1 1 30%;
  margin: 10px;
}
```

---

4. Add hover effect to button

```
button:hover {
  background-color: green;
  color: white;
  transform: scale(1.05);
}
```

---

5. Create a triangle using only CSS

```
.triangle {
```

```
  width: 0;
  height: 0;
  border-left: 50px solid transparent;
  border-right: 50px solid transparent;
  border-bottom: 100px solid red;
}
```

JavaScript Interview Questions

1. What are the different data types in JavaScript?

1. Primitive Data Types (Immutable)

These are the basic, **single-value types**.

EX – Number, String, Boolean, null, Undefined, BigInt

Non-Primitive (Reference) Data Types

These hold **collections** of values or **complex structures**.

EX – array, object, function, `Date`, `Map`, `Set`

JavaScript has:

- **7 Primitive** types: `string`, `number`, `boolean`, `null`, `undefined`, `bigint`, `symbol`
- **1 Complex** type: `object` (which includes arrays, functions, dates, etc.)

2. What is the difference between `var`, `let`, and `const`?
3. What are truthy and falsy values?
4. Explain the difference between == and ===.
5. What is `null` vs `undefined`?

`undefined`

- A variable that has been **declared but not assigned** a value is automatically `undefined`.
- It means the **value is missing** or **not yet defined**.

```
let x;

console.log(x); // undefined

function greet(name) {

  console.log(name);

}

greet(); // undefined
```

`null`

- `null` is an **explicit assignment** by the programmer to indicate **no value** or an intentional **empty object reference**.

```
let user = null;

console.log(user); // null
```

Feature	`undefined`	`null`
Type	`undefined`	`object` (this is a historical bug)
Assigned by	JavaScript engine	Developer (manually)
Meaning	"Not assigned yet"	"Empty" or "No value intentionally"
Use Case	Missing variable or function param	Placeholder for an object or reset
Equality (==)	`null == undefined` → `true`	`true`
Strict Equality (===)	`null === undefined` → `false`	`false`

6. What is hoisting in JavaScript?
7. What is the scope of a variable? (global vs local vs block)

Scope Type	Declared With	Accessible Where
Global Scope	`var`, `let`, `const`	Anywhere in code
Function Scope	`var`, `let`, `const`	Only inside the function
Block Scope	`let`, `const`	Only inside the `{}` block

11. What are JavaScript functions? What are function expressions?
12. What is the difference between `call()`, `apply()`, and `bind()`?

13. What is an arrow function and how is it different from a regular function?

Feature	Regular Function	Arrow Function
Syntax	`function name() {}`	`const name = () => {}`
`this` Binding	Has its own `this`	Inherits `this` from surrounding scope
Hoisting	✅ Yes (function declarations only)	❌ No (behaves like a variable)
Arguments Object	✅ Has `arguments` object	❌ Does NOT have `arguments`
Use in Methods/Constructors	✅ Suitable for both	❌ Not suitable as constructors

14. What is closure in JavaScript?

```
function outer() {
  let name = "Hello";
  function inner() {
    console.log("Welcome, " + name);
  }
  return inner;
}
const greet = outer();
greet();
```

15. What is the event loop and how does JavaScript handle asynchronous code?

The **event loop** is a core part of the JavaScript runtime that allows **asynchronous, non-blocking behavior** — even though JavaScript is **single-threaded**.

It handles:

- Executing code
- Collecting and processing events
- Running queued sub-tasks (like `setTimeout`, `fetch`, or promises)

```
console.log("Start");
setTimeout(() => {
  console.log("Timeout");
}, 0);
Promise.resolve().then(() => {
  console.log("Promise");
});
console.log("End");
```

16. What is the difference between synchronous and asynchronous code?

In JavaScript, a synchronous function is a function that executes its operations sequentially, one after another, in the order they are defined.

```
function add(a, b) {
  return a + b;
}
console.log(add(2, 3)); // 5
console.log("Done");
```

output-

5

Done

Functions running in **parallel** with other functions are called **asynchronous**

```
async function fetchData() {
  return "Data loaded";
}
fetchData().then(result => console.log(result));
```

```
console.log("Done");
```

Output:

```
Done

Data loaded
```

Using await in an async function:

```
async function getData() {
  const response = await new Promise(resolve => {
    setTimeout(() => resolve("Result after 1 second"), 1000);
  });

  console.log(response);
}

getData();
console.log("Next line");
```

output-

```
Next line
Result after 1 second
```

17. What are promises in JavaScript?

```
const myPromise = new Promise((resolve, reject) => {

  setTimeout(() => {

    resolve("Success!");

  }, 1000);

});

myPromise.then(result => {

  console.log(result);  // "Success!" after 1 second

}).catch(error => {

  console.error(error);
```

});

18. What are template literals?
19. What is destructuring in JavaScript?

Destructuring is a JavaScript feature that allows you to **unpack values** from **arrays** or **properties** from **objects** and assign them to **individual variables** in a single, concise statement.

Examples:

Array Destructuring:

```
const colors = ['red', 'green', 'blue'];
const [first, second] = colors;

console.log(first);  // red
console.log(second); // green
```

Object Destructuring:

```
const user = { name: 'Akash', age: 25 };
const { name, age } = user;

console.log(name); // Akash
console.log(age);  // 25
```

The **rest parameter** is used in function definitions to **collect multiple arguments into a single array**. **It is represented by three dots (...).**

Example:

```
function showNames(...names) {
  console.log(names);
}
showNames("Akash", "Priya", "John");
// Output: ['Akash', 'Priya', 'John']
```

Rest with other parameters:

```
function greet(greeting, ...names) {
  names.forEach(name => {
    console.log(`${greeting}, ${name}`);
  });
}

greet("Hello", "Akash", "Priya", "Amit");
```

20. What is the spread operator (. . .) and rest parameter?

The JavaScript spread operator (...) allows us to quickly copy all or part of an existing array or object into another array or object.

Spread with Arrays:

```
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5];  // Spread arr1 into arr2
console.log(arr2); // [1, 2, 3, 4, 5]
```

Spread with Objects:

```
const obj1 = { name: "Akash" };
const obj2 = { ...obj1, age: 25 };
console.log(obj2); // { name: 'Akash', age: 25 }
```

Spread in Function Calls:

```
function sum(a, b, c) {
  return a + b + c;
}
const nums = [1, 2, 3];
console.log(sum(...nums)); // 6
```

21. What are higher-order functions?

`map(), filter(), reduce(), forEach(), sort(), setTimeout(), setInterval()`

A higher-order function is a function that does at least one of the following:

1. **Takes another function as an argument**
2. **Returns a function as its result**

Passing a function as an argument

```
js
CopyEdit
function greet(name) {
  return `Hello, ${name}`;
}

function processUserInput(callback) {
```

```
  const name = "Akash";
  console.log(callback(name));
}

processUserInput(greet); // Hello, Akash
```

Returning a function

```
function multiplier(factor) {
  return function(num) {
    return num * factor;
  };
}

const double = multiplier(2);
console.log(double(5)); // 10
```

22. What is the difference between `map()`, `filter()`, and `reduce()`?

The functions `map()`, `filter()`, and `reduce()` are higher-order functions **in JavaScript (and other programming languages like Python), used for** processing arrays**. Here's a breakdown of each:**

The `map()` method **creates a new array** by applying a **callback function** to **each element** of an existing array, **without modifying** the original array. Original array is **not changed**

Ex- 1

```
a=[1,2,3,4];

b= a.map(x=>x*2);
console.log(b);

Ex-2
const names = ['alice', 'bob', 'charlie'];
const capitalized = names.map(name =>
name.toUpperCase());
console.log(capitalized);
```

Ex- 3

```
 const users = [
  { id: 1, name: 'Rahul', class:"5th" },
  { id: 2, name: 'Priya' },
];
const namesOnly = users.map(user => user.name);
console.log(namesOnly); // ["Akash", "Priya"]

const namesOnly = users.map(user => user.class);
console.log(namesOnly); // ["5th", undefined]
```

The `filter()` method **creates a new array** containing **only the elements** of the original array that **pass a test (return `true`)** specified in a **callback function**. **Does not modify** the original array.

```
a=[1,2,3,4];
b= a.filter(x=>x%2==0);
console.log(b);
```

2. Remove falsy values:

```
const values = [0, "hello", false, "", 42, null];
const truthyValues = values.filter(Boolean);
console.log(truthyValues); // ['hello', 42]
```

3. Get students with marks >= 40:

```
const students = [
  { name: 'Ravi', marks: 35 },
  { name: 'Simran', marks: 65 },
  { name: 'John', marks: 45 }
];
const passed = students.filter(student => student.marks >= 40);
```

```
console.log(passed);
```

The `reduce()` method **reduces** an array to a **single value** by applying a **callback function** to each element, accumulating a result as it goes.

```
const nums = [1, 2, 3, 4];
const sum = nums.reduce((total, n) => total + n, 0);
console.log(sum); // 10


ex- 2.
 const cart = [
  { name: 'Shirt', price: 500 },
  { name: 'Jeans', price: 1200 },
  { name: 'Shoes', price: 2000 }
];
const totalPrice = cart.reduce((sum, item) => sum +
item.price, 0);
console.log(totalPrice); // 3700
```

23. What are `setTimeout` and `setInterval`? How do they work?

```
setTimeout(() => {
  console.log("Hello after 2 seconds");
}, 2000);

setInterval(() => {
  console.log("This runs every 3 seconds");
}, 3000);
```

- **`setTimeout`** can be canceled using `clearTimeout(timerId)`
- **`setInterval`** can be canceled using `clearInterval(intervalId)`

```
const timer = setTimeout(() => {
  console.log("This will not run");
}, 5000);
clearTimeout(timer); // cancels the timeout
const interval = setInterval(() => {
  console.log("This will run every 2 sec");
}, 2000);
setTimeout(() => {
  clearInterval(interval); // stops the interval after 6 seconds
}, 6000);
```

24. What is the prototype chain?

Every JavaScript object **has an internal link to another object called its** prototype**.**

```
function Person(name) {
  this.name = name;
}
Person.prototype.sayHello = function () {
  return "Hello, " + this.name;
};
const john = new Person("John");
console.log(john.sayHello());        // "Hello, John"
console.log(john.toString());        // inherited from Object.prototype
```

25. What is `this` keyword and how does it behave in different contexts?

```
const obj = {
  name: "Akash",
  greet() {
    console.log(this.name);
  }
};
obj.greet(); // "Akash"
```

26. What is currying in JavaScript?

Currying is a functional programming technique where a function with **multiple arguments** is transformed into a sequence of **functions that each take a single argument**.

Example: Without Currying

```
function add(a, b) {
  return a + b;
}
add(2, 3); // 5
```

Example: With Currying

```
function add(a) {
  return function(b) {
    return a + b;
  };
}
add(2)(3); // 5
```

27. What is memoization?

Memoization is a programming technique used to **optimize functions** by **caching their previously computed results**, so that **repeated calls with the same inputs** return the result **instantly from the cache** instead of recalculating it.

```
function memoizedSquare() {
  const cache = {};
  return function(n) {
    if (cache[n] !== undefined) {
      console.log("Fetching from cache...");
      return cache[n];
    }
    console.log("Calculating...");
    const result = n * n;
    cache[n] = result;
    return result;
  };}
const square = memoizedSquare();
console.log(square(5)); // Calculates
console.log(square(5)); // Cached
```

28. Explain debouncing and throttling with examples.
29. What are modules in JavaScript (ES6 `import` / `export`)?
30. What are generators and how do they differ from regular functions?
31. What is async/await and how does it simplify asynchronous code?
32. What are weakMap and weakSet?
33. What is event delegation and why is it useful?

Practical Code-Based Questions

31. Write a function to reverse a string.
32. Write a function to check if a number is prime.
33. Write a function to remove duplicates from an array.
34. Implement a debounce function.
35. Flatten a nested array (`[1, [2, [3, 4]]]` → `[1, 2, 3, 4]`).
36. Write a function to deep clone an object.
37. Explain what happens when you run:

```
console.log(typeof null); // ?
```

38. What will be the output of:

```
js
CopyEdit
console.log([] + []); // ?
console.log([] + {}); // ?
console.log({} + []); // ?
```